

TD 7 — Contribuer efficacement : Bonnes pratiques

Objectifs

- Pratiquer l'écriture de bons messages de commit
 - Rédiger une description de PR de qualité
 - Pratiquer la communication asynchrone
 - Bootstrapper un projet open source avec les fichiers essentiels
 - Comprendre le versionnement sémantique et les release notes
-

Exercice 1 — Réécrire des messages de commit (15 min)

Instructions

Les messages de commit suivants sont mal écrits. Réécrivez-les en suivant les bonnes pratiques (format Conventional Commits).

Mauvais messages à corriger

1. fix bug

→ Votre version : _____

2. updated stuff

→ Votre version : _____

3. WIP

→ Votre version : _____

4. Changed the login function to check for null values and also updated the CSS for the button and fixed a typo in the readme

→ Votre version (peut nécessiter plusieurs commits) :

Commit 1: _____

Commit 2: _____

Commit 3: _____

5. I added a new feature that allows users to export their data to CSV format because they requested it in issue #45

→ Votre version : _____

Exercice 2 — Rédiger une description de PR (20 min)

Contexte

Vous avez développé une fonctionnalité pour un projet de gestion de tâches (todo-list). Votre code :

- Ajoute la possibilité de définir une date d'échéance sur les tâches
- Affiche un badge "En retard" si la date est dépassée
- Ajoute un tri par date d'échéance
- Inclut des tests pour les nouvelles fonctionnalités

L'issue correspondante est la #78 "Add due dates to tasks".

Votre description de PR

Rédigez une description complète en suivant le template ci-dessous.

Titre de la PR

Description

What does this PR do?

Why is this change needed?

Related issue(s)

How to test

1.

2.

3.

Screenshots (if applicable)

Checklist

- [] Tests added/updated
- [] Documentation updated
- [] Code follows project style guidelines

Exercice 3 — Communication asynchrone (15 min)

Instructions

Pour chaque message problématique, identifiez le problème et proposez une meilleure version.

Message 1

@maintainer PING! You haven't answered my issue from yesterday!!!

Problème :

Meilleure version :

Message 2

This doesn't work.

Problème :

Meilleure version :

Message 3

Can someone help me?

Problème : _____

Meilleure version :

Message 4

Your code review is wrong. My approach is better because I've been coding for 10 years and this is how we do it in my company.

Problème : _____

Meilleure version :

Exercice 4 — Bootstrapper un projet open source (25 min)

Instructions

Vous allez créer la structure d'un projet open source fictif (ou réel si vous avez une idée). Le projet est une bibliothèque Python appelée **"timefmt"** qui formate des durées en texte lisible (ex: "2h 15min 30s").

Étape 1 — Créer le dépôt

Créez un nouveau dépôt sur GitHub (ou localement) et initialisez-le avec les fichiers suivants.

Étape 2 — README.md

Rédigez un README.md qui contient **au minimum** :

1. **Nom et mission en une phrase** (ex: "timefmt — Human-readable time duration formatting for Python")
2. **Résumé en un paragraphe** : quel problème ça résout, pour qui
3. **Installation** (même fictive : `pip install timefmt`)
4. **Quick start** avec un exemple de code
5. **Licence** (indiquer laquelle)

timefmt

Installation

Quick start

License

Étape 3 — Choisir une licence

Choisissez une licence pour votre projet. Justifiez votre choix.

Licence choisie : _____

Justification :

Rappel : si vous hésitez, MIT est un bon point de départ. Vous pouvez toujours aller vers plus restrictif par la suite.

Étape 4 — CONTRIBUTING.md

Rédigez un guide de contribution minimal contenant :

1. Comment reporter un bug
2. Comment proposer une fonctionnalité
3. Comment soumettre une PR (format de commit, tests attendus)
4. Standards de code (ex: Black, PEP 8)

Étape 5 — Évaluation croisée

Échangez vos dépôts avec un binôme. Évaluez le projet de votre voisin :

- ☐ Le README donne-t-il envie de tester le projet ?
- ☐ La licence est-elle clairement indiquée ?
- ☐ Le CONTRIBUTING.md est-il suffisamment clair pour un nouveau contributeur ?
- ☐ Manque-t-il un fichier essentiel (.gitignore, CODE_OF_CONDUCT) ?

Feedback :

Exercice 5 — Versionnement et Release Notes (15 min)

Partie A — Semver

Pour chacun des changements suivants, indiquez si la version suivante devrait incrémenter le numéro MAJEUR, MINEUR ou CORRECTIF. La version actuelle est **1.4.2**.

1. Correction d'un bug : la fonction `format_duration()` retournait "0s" pour les durées de moins d'une seconde au lieu de "< 1s".

→ Nouvelle version : _____ (justification : _____)

2. Ajout d'une nouvelle fonction `format_duration_short()` qui retourne un format compact (ex: "2h15m" au lieu de "2 hours 15 minutes"). Les fonctions existantes ne changent pas.

→ Nouvelle version : _____ (justification : _____)

3. Renommage du paramètre `lang` en `locale` dans toutes les fonctions publiques. L'ancien paramètre n'est plus accepté.

→ Nouvelle version : _____ (justification : _____)

4. Mise à jour d'une dépendance interne (pas de changement d'API, pas de bug visible corrigé).

→ Nouvelle version : _____ (justification : _____)

Partie B – Rédiger une release note

Rédigez une release note au format “Keep a Changelog” pour une version fictive de timefmt qui regroupe les changements 1, 2 et 4 ci-dessus.

[__.__.__] - 2026-03-25

Added

Fixed

Changed

Pour aller plus loin

Ressources

- **Conventional Commits** : <https://www.conventionalcommits.org>
- **How to Write a Git Commit Message** : <https://cbea.ms/git-commit/>
- **Thoughtful Code Review** : <https://github.com/google/eng-practices/blob/master/review/>
- **Producing Open Source Software** : <https://producingoss.com/> (Karl Fogel)
- **Semantic Versioning** : <https://semver.org>
- **Keep a Changelog** : <https://keepachangelog.com>

Outils

- **commitlint** : Vérifie le format des commits
- **husky** : Git hooks pour automatiser les vérifications
- **semantic-release** : Release automatique basée sur les commits
- **cookiecutter** / **copier** : Templates pour bootstrapper un projet

Préparation séance 8

1. Comment sont prises les décisions dans le projet Linux ?
2. Qu’est-ce que le “consensus paresseux” ?
3. Qu’est-ce qu’une fondation open source (Apache, Linux Foundation) ?